

Lesson 6: Reinforcement Learning Foundations

Lesson 6: Reinforcement Learning Foundations Q-learning, DQN, exploration, rewards, and optimization intuition

Dr. Syed Muzamil Basha

School of Computer Science and Engineering, REVA University

Learning Outcomes

- Explain state, action, reward and policy.
- Understand Q-learning at an intuitive level.
- Explain the role of alpha, gamma and epsilon.
- Describe the DQN architecture in the notebook.

Reinforcement Learning in One Picture

State $\rightarrow$ Action $\rightarrow$ Reward $\rightarrow$ Next State

- An agent interacts with an environment.
- It learns which actions lead to better long-term rewards.
- Unlike supervised learning, there is no fixed label for every state.

Q-Learning Setup

```
env = gym.make("FrozenLake-v1", is_slippery=False)
state_size = env.observation_space.n
action_size = env.action_space.n
Q_table = np.zeros((state_size, action_size))
alpha = 0.1
gamma = 0.99
epsilon = 1.0
epsilon_decay = 0.995
epsilon_min = 0.01
episodes = 2000
```

- FrozenLake supplies a discrete state and action space.
- The Q-table stores a value for each state-action pair.
- Alpha controls the update size.
- Gamma controls the importance of future reward.
- Epsilon controls exploration.

Interpreting the Bellman Update

$$Q(s, a) \leftarrow Q(s, a) + \alpha[r + \gamma \max_{a'} Q(s', a') - Q(s, a)]$$

- The target combines immediate reward and discounted future value.
- The difference between target and current Q-value is the learning signal.
- Repeated interaction gradually improves the table.

Interpreting Exploration vs Exploitation

- With high epsilon, the agent tries random actions.
- As epsilon decays, the agent increasingly uses the best-known action.
- This balances learning new possibilities with using current knowledge.

DQN: Replace the Table with a Neural Network

```
class DQN(nn.Module):  
    def __init__(self, state_size, action_size):  
        super(DQN, self).__init__()  
        self.fc1 = nn.Linear(state_size, 24)  
        self.fc2 = nn.Linear(24, 24)  
        self.fc3 = nn.Linear(24, action_size)  
    def forward(self, x):  
        x = torch.relu(self.fc1(x))  
        x = torch.relu(self.fc2(x))  
        return self.fc3(x)
```

- The network maps a state vector to one value per action.
- Two hidden layers use ReLU activation.
- The final layer has one output for each possible action.
- This allows learning in larger or continuous state spaces where a Q-table can become impractical.

DQN Memory and Replay

- The notebook stores experiences in a replay memory.
- A minibatch is sampled from memory for training.
- Each experience contains state, action, reward, next state and done flag.
- Replay breaks strong correlations between consecutive experiences.

Interpreting the Recorded Output

Episode 1: 18.0 Episode 10: 15.0

- The notebook prints total reward for each episode.
- The recorded values fluctuate, which is normal during exploration.
- The file ends with `Training complete!`; it does not provide a final aggregate performance metric.

- Reinforcement learning introduces iterative optimization and reward-driven search.
- Quantum algorithms such as Grover also use structured search, but they operate on quantum states instead of an RL environment.
- Optimization concepts reappear in variational quantum algorithms.

- Q-learning stores action values in a table.
- DQN approximates action values with a neural network.
- Exploration and exploitation must be balanced.
- The notebook demonstrates both a tabular and deep RL workflow.

**Reinforcement learning teaches a different learning paradigm:
learn by interacting with an environment.**

Quick Check

- ① What is a state?
- ② Why use a discount factor?
- ③ What does epsilon control?
- ④ Why is a replay buffer useful?